תכנות מערכות מתקדם - תרגיל מספר 3

בתרגיל זה שני חלקים

<u>חלק א</u>

בחלק זה עליכם להמשיך להשתמש בפרוייקט ה JIRA מתרגילים 1,2, ולתכנן בו את חלק ב של תרגיל זה.
תהליך הפיתוח של חלק ב יהיה בצמוד לפרוייקט הJIRA, ואתם חייבים להשתמש ב JIRA **<u>לאורך כל הדרך בזמן אמת</u>**. הדבר ייבדק על ידי הבודק.
אם ברצונכם להמשיך להשתמש ב Github מהתרגילים הקודמים אתם יכולים לעשות זאת במספר דרכים.
דרך אחת היא לבצע fork לפרוייקט.
דרך אחרת היא ליצור branch נפרד עבור תרגיל 2 ולא לגעת בו (ב branch הנפרד) - כדי שהבודק יוכל לבדוק אותו.
ואז אתם ממשיכים לעבוד על ה main branch בשביל תרגיל 3.
לחילופין, אתם יכולים לא לגעת ב main branch וליצור branch חדש שיהיה ה main branch עבור תרגיל 3.
בכל מקרה, שימו לב לא לגעת בbranch שמיועד לתרגילים 1,2 לאחר מכן - כדי לא לפגוע בימי ה"חסד".
אם אתם רוצים, אתם יכולים ליצור repository נפרד עבור תרגיל 3.
אם החלפתם קבוצה ואין באפשרותיכם להשתמש בעקבות כך בפרויקט ה JIRA (או Github וכו') מהתרגילים הקודמים, צרו פרוייקטים חדשים בהתאם עם הקוד מהתרגילים הקודמים.

בפרט, עליכם להקפיד על:

- חלוקה ל epics
- כתיבת users stories
- כתיבת tasks
- יצירת ספרינט, מינוי scrum master מבין חברי הקבוצה, תכנון הספרינט, קיום מפגשי סטטוס לאורך הספרינט (לא כל יום, אבל לפחות פעמיים בשבוע) חובה לתעד ב JIRA את כל הפעולות הללו ואת התוצאות שלהן. למשל, התובנות והמסקנות של מפגשי הסטטוס חייבים להיות מפורטים ומתועדים.
- עדכון סטטוס משימות (in progress, code review, done) כולל עדכון של הactivity של משימות **<u>תוך כדי</u>** ביצוע המשימה. שימו לב, עליכם להוסיף את הסטטוס code review, הוא לא מגיע ברירת מחדל. משימה עוברת ל code-review כאשר סיימתם אותה ופתחתם לה pull request. באפשרותכם כעת לשייך את המשימה ב JIRA אל חבר צוות אחר שצריך לעשות review לקוד שלכם. לחילופין, תוכלו להשאיר את המשימה אצלכם ול"תייג" את החבר צוות האחר ב activity של הissue ב JIRA. בכל מקרה, עליכם להגדיר בצורה ברורה מי לבסוף מעביר את המשימה ל done.
- שיוך משימות לסטודנט שמבצע אותן **<u>לפני שהוא מתחיל לעבוד עליהן</u>**.

- עליכם לזהות אילו משימות "תוקעות" משימות אחרות (is blocked by) ולהביא את זה לידי ביטוי בפיצ'ר הרלוונטי בJIRA.
- שיוך **כל** המשימות ל feature branches. כלומר, יש ל Github ול JIRA התממשקות כך שאם רושמים את שם הJIRA issue בשם של ה branch והמשימות, JIRA יודע להציג את זה בJIRA issue אוטו'.

חלק ב

בתרגיל זה נרצה לבנות שרת אינטרנטי בארכיטקטורת MVC, שיהווה את ליבת האפליקציה שלנו.

השרת שלנו יתמוך בפונקציונאליות של אפליקציית Google Drive ויתן מענה למסכים הבאים:
- מסך התחברות.
- מסך הרשמה (אצלנו מספיק שהוא יהיה דומה למסך ההתחברות רק עם שדות מתאימים. כלומר כשנרשמים צריך לתת נתונים שלא נותנים בעת התחברות, כמו שם, תמונה וכו').
- מסך ראשי לאחר התחברות.

ההמלצה היא להסתכל על המסכים הללו כשמתכננים את השרת - כי זה יאפשר להבין איזה שדות צריכים להיות באובייקטים השונים שהשרת צריך.

בתרגיל זה אינכם מממשים את המסכים עצמם - אלא רק את פונקציונאליות צד השרת שלהם, כפי שמתוארת בהמשך.

כעת נתחיל לפרט את ה API שעליכם לממש בשרת.

התחברות והרשמה:
- חלק מהפעולות מול השרת יכולות להתבצע רק על ידי משתמש שרשום למערכת והזדהה מולה.

כיצד בא לידי ביטוי בAPI?
עבור הכתובת: http://foo.com/api/users
- פעולת Post יוצרת משתמש חדש (הטופס של מסך התחברות שולח ב body את המידע של המשתמש כ json)

עבור הכתובת: http://foo.com/api/users/:id
- פעולת Get נותנת את הפרטים של המשתמש עם המזהה id: (שם, תמונה וכו')
- עליכם להבין בעצמכם אילו שדות אמורים להיות ל user.

עבור הכתובת: http://foo.com/api/tokens
- פעולת Post מקבלת את השם משתמש והסיסמא (שוב, json ב body) ובודקת אם המשתמש רשום למערכת או לא. בתרגיל הזה הפלט json יכיל את מזהה המשתמש אם קיים כזה משתמש או הודעת שגיאה מתאימה אם לא - אבל זה בכל מקרה ישתנה בתרגיל הבא.

- לכל פניה לשרת עליכם לחשוב האם היא יכולה להתבצע על ידי משתמש שאינו מחובר למערכת או היא דורשת משתמש מחובר. בתרגיל הבא תתבקשו לאכוף זאת. בתרגיל זה אתם פשוט מתבקשים להעביר כ http header את מזהה המשתמש של משתמש מחובר לכל בקשה שדורשת התחברות.

בסוף התרגיל יש דוגמאות לעבודה מול API. (שם לעומת זאת לא תראו את ה http header הנ״ל - למרות שהוא אמור להיות)

<u>עמוד לאחר התחברות:</u>

המשתמש רואה את רשימת הקבצים והתיקיות שלו.

<u>כיצד בא לידי ביטוי בAPI?</u>

עבור הכתובת: http://foo.com/api/files
- פעולת Get תחזיר את רשימת כל הקבצים והתיקיות של המשתמש ברמה העליונה ביותר (כלומר ב /).
- פעולת Post תיצור קובץ/תיקייה חדש/ה.

עבור הכתובת: http://foo.com/api/files/:id
- פעולת Get נותנת את הפרטים של הקובץ/תיקייה שהמזהה שלו זה id:
- פעולת PATCH עורכת קובץ/תיקייה קיים id:
- פעולת DELETE מוחקת קובץ/תיקייה קיים id:

עבור הכתובת: http://foo.com/api/files/:id/permissions
- פעולת Get נותנת את ההרשאות של הקובץ/תיקייה שהמזהה שלו זה id:
- פעולת Post יוצרת הרשאות עבור הקובץ/תיקייה שהמזהה שלו הוא id:

עבור הכתובת: http://foo.com/api/files/:id/permissions/:pId
- פעולת Patch מעדכנת את ההרשאות של pId:
- פעולת Delete מוחקת את ההרשאות של pId:

עבור הכתובת: http://foo.com/api/search/:query
- פעולת Get נותנת את הקבצים/תיקיות שהם תוצאת החיפוש של query:, כלומר, קובץ/תיקייה שבשמם או בתוכן שלהם מוכלת המחרוזת query:

שימו לב, הפעולות הנ״ל דורשות לעיתים לפנות מול השרת של תרגיל 2.
למשל, יצירת קובץ, שליפת קובץ, חיפוש בקובץ ועוד.

כדי לממש את הפעולות מול השרת מתרגיל 2, שרת ה web שלכם יגש לשרת מתרגיל 2. כלומר, השרת שלכם יפתח סוקט **כלקוח** מול השרת של תרגיל 2.

חובה לוודא שהפעולות שמוגדרות בתרגיל ממומשות כפי שהוגדרו. הקפידו להחזיר את סטטוס ה http המתאים בתשובות (200, 201, 204, 404 וכו׳)

שימו לב, מכיוון שהשרת שלנו חושף WEB SERVICE, כל הפניות ל /api/ מחזירות JSON ולא HTML.
באחריותכם לוודא ש״נעלתם״ את קוד השרת שאתם מגישים לתרגיל 2 ב branch מתאים, שהבודק יוכל לבדוק אותו - וכדי שלא תתערבב העבודה על תרגיל 3 עם מה שהוגש בתרגיל 2. עליכם להסביר זאת בצורה ברורה ב README.

עליכם לשנות את המימוש של השרת מתרגיל 2 כך שה thread ים ינוהלו בעזרת thread pool ולא באמצעות thread נפרד לכל לקוח.

הנחיות:
עליכם ליצור שרת NodeJS בארכיטקטורת MVC בעצמכם אשר יחשוף את אותה הפונקציונאליות המתוארת לעיל. המידע נשמר בשרת ״in-memory״, כלומר, פשוט ברשימות נדיפות כך שאיתחול השרת מחדש מוחק את כל המידע שהוזן עד כה (בשרת האינטרנט, כי בשרת מתרגיל 2 זה שמור בקבצים).
על השרת לממש את ה RESTful API שתואר לעיל.
שימו לב שבגלל שאנחנו מממשים api, ה View אינו רלוונטי (במובן הקלאסי של להחזיר html), כי אנחנו מחזירים json ישירות.

יש להמשיך ולתכנן את הקוד בעזרת החומר שנלמד ובדגש באופן שיאפשר הרחבה עתידית של הקוד. בפרט, יש לתת דגש על loose coupling ועל עקרונות ה SOLID. שימו לב, שבתרגילים הבאים תתבקשו לתחזק את הקוד הזה על ידי שינוי ותוספות שתתבקשו לממש. מימוש נכון, יביא לכך שיישום של הדרישות החדשות שיהיו יהיה יחסית פשוט למימוש.

יש להקפיד על קוד נקי, מסודר ו**מתועד**.

השרת שאתם תכתבו חייב לטפל בכל מקרי הקצה השונים.
אופן המימוש של קוד השרת יכול (**מאחורי הקלעים**) להכיל כל DESIGN של אובייקטים וקשרים ביניהם, **לאו דווקא** מה שהAPI מגדיר. ועדיין, עליכם לתמוך בAPI הזה **במדויק**.
תוכלו להוסיף לשרת שלכם יכולות נוספות אם תרצו - אבל אתם חייבים לתמוך בAPI **הזה** במדויק.

אין לאחסן (או לשלוח) מידע חשוב (כמו סיסמאות אמיתיות או טקסט חשוב) אל השרת או בשרת או בשום מקום, כי התרגיל לא לוקח בחשבון דגשים רבים שיש לקחת בהקשר של ה security של האפליקציה.

מותר להשתמש ב NodeJS ו Express בלבד.

אני מזכיר, אין להעלות שום סיסמאות, מידע פרטי, רגיש או סודי, מפתחות, טוקנים, API Keys וכו׳ וכו׳ וכו׳ בשום מקום בפרויקט ובקורס. לא בgithub, לא בקוד, לא בREADME, לא ב WIKI לא בJIRA או בDOCKER או ב DB ולא בשום מקום אחר. זיכרו שלרוב הדברים יש היסטוריה - והמידע מופיע גם שם, אפילו אם מחקתם אותו.

חובה להגיש את התרגיל בזוגות או בשלשות. הגשה ביחידים (מכל סיבה שהיא), תגרור תקרה של ציון מקסימלי של 75 בכל תרגיל שיוגש ביחידים. לא ניתן יהיה לקבל ציון מעל 75 בתרגיל שהוגש ביחידים.

חובה לעבוד בגיט לאורך כל התרגיל. כל סטודנט עובד מהמחשב שלו ומהמשתמש הנפרד שלו. לא ניתן לעבוד ביחד מאותו מחשב או מאותו משתמש או ללא גיט. חובה לעבוד בשיטת feature branches בלבד ולתחזק branch ראשי שאליו ממרג׳ג׳ים את הfeature branches. חובה למרג׳ג׳ קוד לbranch הראשי על ידי pull request בלבד, וחובה שמאשר הpull request יהיה שונה ממי שביקש את ה pull request (במידה כמובן שאכן הבקשה אושרה - ייתכן ומי שבוחן את בקשת הpull ימצא בעיות במימוש שיש לתקן טרם אישור הבקשה). כל pull request חייב לעבור אישור של כל שאר חברי הקבוצה.
כלומר, אם אתם זוג - אחד פותח pull request <u>והשני עובר על הקוד **ונותן הערות** (אם ישנן) **ורק אם הקוד תקין**</u> מאשר את הrequest. אם אתם שלשה - אחד פותח pull request והשניים האחרים צריכים לעבור על הקוד ונותנים הערות (אם ישנן) **<u>ורק אם הקוד תקין</u>** שניהם מאשרים את הrequest. דרך אגב, לgithub יש פיצ׳ר מובנה שמאפשר לעשות את זה. אפשר להשתמש בו אבל לא חובה. (הוא זמין בחינם רק למי שעשה את החשבון כחשבון סטודנט, ואנחנו משתמשים **<u>רק</u>** בכלים חינמיים). אם עשיתם חשבון מתאים, אתם יכולים להשתמש בזה וזה פשוט לא יאפשר למי שפתח את הpull request לאשר לעצמו. אם בחרתם שלא להשתמש בפיצ׳ר המובנה, באחריותכם להקפיד לבצע את הלוגיקה שהוגדרה (שאחד פותח PR והאחרים מאשרים).

(למרג׳ג׳ == לעשות merge)

חובה לתכנן ולחלק את המשימות בJIRA כספרינט, ולתחזק אותן בהתאם להתקדמות בפרויקט. אלה דרישות מהותיות ומשקלם בתרגיל משמעותי.

על ההגשה להכיל Dockerfile מתאים והוראות הרצה שלו כך שניתן יהיה להריץ בעזרת Docker את התרגיל, כלומר שרת web בנפרד והשרת של תרגיל 2 בנפרד. ניתן להשתמש ב docker-compose למעוניינים.

את התרגיל יש להגיש למודל לתיבת ההגשה על ידי הגשת קובץ טקסט בשם details.txt עם שמות ות.ז. של המגישים וקישור לגיט. שימו לב, חובה על הקובץ להיות בפורמט הבא:
Israel Israeli 123456789

Israela Israeli 012345678
Israeli Israeli 012345679
LINK TO GITHUB HERE

בלי רווחים נוספים, בלי שורות נוספות, ובשפה האנגלית בלבד. אי הגשה של קובץ ה details.txt הנ״ל או הגשתו באופן שונה ממה שהוגדר, **תגרור הורדה של 20 נקודות בציון התרגיל.**

חובה להגיש את קובץ ה details למודל מיד עם תחילת העבודה. (משימה בJIRA)

אנא דאגו לכך שהגיט שלכם יהיה במצב private לכל אורך הסמסטר. יש להוסיף את המייל של המתרגלים (המייל המשותף שפירסמו לכם, לא המיילים הפרטיים שלהם) עם הרשאות גישה ל Github ול JIRA.

אסור להשתמש בשום סיפרייה/מודול שלא הופיע במצגות הקורס או קיבל אישור מפורש לשימוש בפורום הקורס. **אין** לבקש אישור מילולי או במייל מהמרצה/מתרגלים - רק דרך הפורום. לא צריך לבקש אישור על ספריות פשוטות ומובנות בשפה. עם זאת, אין להשתמש בשום דבר שאינו טריוואלי. למשל, שימוש במבנה נתונים, כמו list הוא גם פשוט וטריוויאלי וגם מובנה בשפה, כך שאין בעיה להשתמש בו. עם זאת, שימוש בספרייה שמנהלת מיילים, למשל, אינה דבר טריוויאלי ולכן אסורה לשימוש, ובפרט, במקרים שמדובר במשהו שאתם חייבים לממש בעצמכם אזי אסור להשתמש בספרייה במקום לממש בעצמכם.

אל תעלו לגיט את תיקיית ה node_modules.
שאלות לגבי התרגיל יש לשאול בפורום בלבד.

קובץ ה README צריך להכיל הסברים מתאימים אודות האפליקציה, תמונות מתאימות, והסבר של איך מריצים ודוגמאת הרצה.
שימו לב, אין לשנות את הrepo לאחר ההגשה. כל שינוי לאחר ההגשה ייחשב כאיחור.

עבודה עצמית בלבד. הנושא ייבדק ויאכף. שימו לב, התרגיל הוא שלכם ואתם צריכים לממש אותו ולא כלים מבוססים Gen AI. שימוש בכלים מסוג זה באופן שכלים אלו מממשים עבורכם חלקים מהתרגיל שקול לעבודה שאינה עצמית ועצמאית. הנושא ייבדק וייאכף.

בהצלחה!

נספח:

נדגים עבודה מול השרת בעזרת הדוגמא הבאה (הדוגמא להמחשה בלבד. ייתכן שצריך שדות נוספים, תרחישים נוספים וכו׳. כמובן שפה אני מדגים רק את התוויות ויש עוד הרבה endpoints אחרים שהתרגיל מבקש לממש והם לא מופיעים פה):

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 2
Connection: keep-alive
Keep-Alive: timeout=5

[]
```

פקודה שהרצתי בטרמינל: (שימו לב: בנספח אני לא ארשום את כל השדות של קובץ/תיקייה. זה חלק ממה שאתם צריכים לחשוב עליו. אני ארשום רק את שם הקובץ, אתם צריכים להשלים את שאר השדות.)

```
curl -i -X POST http://localhost:3000/api/files \
-H "Content-Type: application/json" \
-d '{"name": "aaa"}'
```

פלט שקיבלתי: (שימו לב, ייתכן שאתם תבחרו id שונה בפורמט שלו)

```
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/files/ffc76e45-2353-4265-9e6a-29760a01f408
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 0
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 103
Connection: keep-alive
Keep-Alive: timeout=5

[
  {
    "id": "ffc76e45-2353-4265-9e6a-29760a01f408",
    "name": "aaa"
  }
]
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files/ffc76e45-2353-4265-9e6a-29760a01f408
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 89
Connection: keep-alive
Keep-Alive: timeout=5

{
  "id": "ffc76e45-2353-4265-9e6a-29760a01f408",
  "name": "aaa"
}
```

פקודה שהרצתי בטרמינל:

```
curl -i -X POST http://localhost:3000/api/files \
-H "Content-Type: application/json" \
-d '{"name": "bbbb"}'
```

פלט שקיבלתי:

```
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /api/files/5e3d043c-5666-4c6e-8a52-b389765ba158
Connection: keep-alive
Keep-Alive: timeout=5
Content-Length: 0
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 206
Connection: keep-alive
Keep-Alive: timeout=5

[
  {
    "id": "ffc76e45-2353-4265-9e6a-29760a01f408",
    "name": "aaa"
  },
```

```
  {
    "id": "5e3d043c-5666-4c6e-8a52-b389765ba158",
    "name": "bbbb"
  }
]
```

פקודה שהרצתי בטרמינל:

```
curl -i -X PATCH
http://localhost:3000/api/files/5e3d043c-5666-4c6e-8a52-b389765ba158 \
-H "Content-Type: application/json" \
-d '{"name": "cccccc"}'
```

פלט שקיבלתי:

```
HTTP/1.1 204 No Content
X-Powered-By: Express
Connection: keep-alive
Keep-Alive: timeout=5
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 218
Connection: keep-alive
Keep-Alive: timeout=5

[
  {
    "id": "ffc76e45-2353-4265-9e6a-29760a01f408",
    "name": "aaa"
  },
  {
    "id": "5e3d043c-5666-4c6e-8a52-b389765ba158",
    "name": "cccccc"
  }
]
```

פקודה שהרצתי בטרמינל:

```
curl -i -X PATCH http://localhost:3000/api/files/foo \
-H "Content-Type: application/json" \
```

```
-d '{"name": "cccccc"}'
```

פלט שקיבלתי:

```
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 35
Connection: keep-alive
Keep-Alive: timeout=5

{
  "error": "File not found"
}
```

פקודה שהרצתי בטרמינל:

```
curl -i -X POST http://localhost:3000/api/files \
-H "Content-Type: application/json" \
-d '{}'
```

פלט שקיבלתי:

```
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 33
Connection: keep-alive
Keep-Alive: timeout=5

{
  "error": "Name is required"
}
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 218
Connection: keep-alive
Keep-Alive: timeout=5

[
  {
    "id": "ffc76e45-2353-4265-9e6a-29760a01f408",
```

```
    "name": "aaa"
  },
  {
    "id": "5e3d043c-5666-4c6e-8a52-b389765ba158",
    "name": "cccccc"
  }
]
```


פקודה שהרצתי בטרמינל:

```
curl -X DELETE http://localhost:3000/api/files/ffc76e45-2353-4265-9e6a-29760a01f408
```

פלט שקיבלתי:

```
HTTP/1.1 204 No Content
X-Powered-By: Express
Connection: keep-alive
Keep-Alive: timeout=5
```

פקודה שהרצתי בטרמינל:

```
curl -i http://localhost:3000/api/files
```

פלט שקיבלתי:

```
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 117
Connection: keep-alive
Keep-Alive: timeout=5

[
  {
    "id": "5e3d043c-5666-4c6e-8a52-b389765ba158",
    "name": "cccccc"
  }
]
```

שימו לב, אני לשם הפשטות השתמשתי ב curl. אתם יכולים להשתמש בכלים אחרים מתקדמים יותר עם ממשקים גרפיים אם אתם מעוניינים.